



Algorithm & C-Programming

Final exam:
Bachelor B1 (School year 2028)

The use of calculators and any electronic device is prohibited.

If, during the exam, a candidate notices what appears to be an error in the wording, they must mention it in their answer sheet and continue the exam, explaining the reasoning behind any decisions they choose to make.



Infos

Edition:

1.0

 (2025.12.10)

svandamme@omnesintervenant.com



00. FOREWORD

A. General information

Quick reminder: it is always worth reading the entire wording before you start.
Identify where you can earn points efficiently and quickly.

It is highly recommended to group the questions of the same exercise together, instead of jumping back and forth between exercises.

Keep a clear structure using the exercise headers, and leave some space for possible corrections later.

Example:

Exercise 1

Q. 1) Answer 1, blabla (...)

Q. 2) // I don't know the answer.

Q. 3) (space for later)

Q. 4) Answer 4, blabla (...)

Even if you cannot answer a question immediately, leave room for it later — like Q.3 in the example.

Grading scales may be slightly adjusted during correction.

This exam was designed to be about: **1h30** long.

Good luck with the exam. :)

B. Prerequisite chapters

For this exam, the following chapters are relevant:

Basic of C programming (variable, type...)

Arrays and matrixes

Pointers

Debugging/Common errors

Control flow

Algorithms reasoning

Functions



01. Observation points (15/100 points)

You are given the following program, which contains three display points (*printf()* calls).

Main - C

```
int main()
{
    int A = 3, B = 7, C ;
    int *pt_1 = &A ;
    int *pt_2 = &C ;

    C = A + B ;
    *pt_1 += 3 ;

    /// (1)
    printf("A = %i, B = %i, C = %i\n", A, B, C) ; // >> (1)st OBSERVATION POINT

    A++ ;
    B = A + 6 ;
    C = f1(pt_1, B) ; // NOTE: There is an observation point INSIDE...

    pt_2 = pt_1 ;
    pt_1 = &B ;

    *pt_1 = 6 ;
    *pt_2 = 3 ;

    /// (3)
    printf("A = %i, B = %i, C = %i\n", A, B, C) ; // >> (3)rd OBSERVATION POINT

    return 0 ;
}
```

Function: f1 - C

```
long f1(int *A, int B)
{
    int C = 4 ;

    B = *A + 5 + C ;
    *A = 8 * 2 ;

    /// (2)
    printf("A = %i, B = %i, C = %i\n", *A, B, C) ; // >> (2)nd OBSERVATION POINT

    return B ;
}
```

Q.1) Write what the program will display in the terminal at each observation point (1), (2), and (3), as indicated in the code.

Q.2) Draw a simple diagram showing which pointer points to which variable (A, B, C) at observation point (1) and at observation point (3).

02. Array works (20/100 points)

You are in charge of developing the train ticket payment system for the upcoming winter holidays.

Depending on the age of the traveler, the price of a ticket is not the same for everyone:

- **Less than 3 years old:** the ticket is free
- **Less than 10 years old:** the ticket is half the price
- **Less than 27 years old OR older than 70 years old:** 10% reduction.

In all other conditions, the ticket is paid the full price.

Q.1) Write the function *price(...)* that returns the final ticket price.

Function template: **price** - C

```
// Parameters:  
//   age - age of the traveler  
//   basePrice - base ticket price (in €)  
// Returns:  
//   The final price after reductions  
float price(int age, float basePrice)  
{  
    ...  
}
```

Q.2) Write a *main()* function that tests your function *price(...)*.

You can, *for example*, imagine a *complete* fictional traveler that want to go from Paris to Nice (in the south of France) for their holidays.

Here are the parameters you can use:

- Age of the traveler: **25 years old**
- Base ticket price: **129.99 €**

Display the final price the traveler will have to pay, using *printf()* in the *main()*.





03. Fix the codes (20/100 points)

For each code snippet, find the errors that prevent the program from compiling or giving the correct result.

Q.1) Temperature Converter (°C to °F):

Find the error #1 - C

```
#include <stdio.h>

float convertCtoF(float tempC)
{
    float tempF ;
    tempF = (9/5) * tempC + 32.0 ;

    return tempF
}

int main(void)
{
    float c, f ;
    printf("Enter temperature in Celsius : ");
    scanf("%f", c);

    f = convertCtoF(c);
    printf("Result = %f\n");

    return 0 ;
}
```

There are at least 4 errors.

Q.2) Odd/Even checker

Find the error #2 - C

```
#include <stdio.h>

void checkNumber(int n)
{
    if (n % 2 = 0)
        printf("%d is odd\n", n);
    else
        printf("%d is even\n", n);
}

int main()
{
    int val = 7;
    checkNumber(val);
    return 0;
}
```



Q.3) Array maximum finder:

Find the error #3 - C

```
int findMax(int arr[], int size)
{
    int max = arr[1];
    for (int i = 1 ; i <= size ; i++)
        if (arr[i] > max)
            max = arr[i];

    return max;
}

int main()
{
    int values[5] = {4, 9, 1, 6, 2};
    int m = findMax(values, 5);
    printf("Max value is : %d\n", max);

    return 0;
}
```



04. Matrix Edge Calculator (25/100 points)

You're given a $M \times N$ matrix of integers ($\forall (M, N) \in (\mathbb{N}^*)^2$).

You want to calculate the sum of the values located in the outer border of the matrix.

For example, in the following 3×5 matrix, the sum corresponds to the integers shown in blue:

$$\begin{vmatrix} 2 & -4 & 1 & 0 & 1 \\ 5 & -11 & 8 & 3 & -5 \\ 7 & -4 & 3 & 10 & 6 \end{vmatrix} > \begin{vmatrix} \color{blue}{2} & \color{blue}{-4} & \color{blue}{1} & \color{blue}{0} & \color{blue}{1} \\ \color{blue}{5} & -11 & 8 & 3 & \color{blue}{-5} \\ \color{blue}{7} & \color{blue}{-4} & \color{blue}{3} & \color{blue}{10} & \color{blue}{6} \end{vmatrix} > \text{outlineValue} = \color{blue}{2} - \color{blue}{4} + \color{blue}{1} + \dots = 22$$

To help you, you're reminded that the values are in the:

- FIRST and LAST **rows (M)**
- FIRST and LAST **columns (N)**

Q.1) Write the function:

Function: **outlineValue** - C

```
int outlineValue(int matrix[][N], int m, int n)
{
    ...
}
```

that calculates the outlineValue of a given matrix.

INFO: in the function signature, **N is a constant** representing the number of columns. You may assume it is defined to the right size of the given matrix beforehand, using:

```
#define N 10
```

for a 4×10 matrix (as an *example*).



05. Mystery function (20/100 points)

You are given the following mystery function:

Function : **mystery** - C

```
int mystery(int a, int b)
{
    int x = 0;
    int y = a;

    while (y >= b) {
        int p = b;
        int k = 1;

        while (p + p <= y) {
            p = p + p;
            k = k + k;
        }

        y = y - p;
        x = x + k;
    }

    return x;
}
```

Q.1) Explain the principles of this algorithm.

What does the function compute? What is its purpose?

You may illustrate your explanation with examples, diagrams, or small computations.

Q.2) What is the result of the function call: “**mystery(37, 6)**”?